

Answer Set Programming

Intelligenza Artificiale e Laboratorio

Introduzione

- * Inizialmente: “War of semantics” per dare una semantica alla negazione per fallimento adottata dagli interpreti Prolog
- * Presto divenuta fondamento di una nuova tecnica di programmazione
 - * Answer set programming
 - * Le soluzioni sono i **modelli** (Answer Sets), non più le prove
 - * inferenza non basata su backward chaining, ma sulla costruzione di modelli stabili

Introduzione

- * ASP:

- * particolarmente utile per risolvere problemi combinatori (es. soddisfacimento di vincoli, planning)
- * ASP solvers molto efficienti sviluppati per supportare questa metodologia (DLV, smodels, **CLINGO**, Cmodels, ...)

Codice ASP

- * Insieme finito di regole

$a:- b_1, b_2, \dots, b_n, \text{not } c_1, \text{not } c_2, \dots, \text{not } c_m$

- * a, b_i, c_j sono letterali della forma p o $\neg p$

- * $\neg$ è la negazione classica

- * not è la negazione per fallimento

- * a è opzionale, senza ho un **integrity constraint**:

$:- a_1, a_2, \dots, a_k$

- * è inconsistente che $a_1, a_2, \dots, a_k$ siano tutti veri

Codice ASP

* ASP:

- * si applica ai soli programmi logici proposizionali
- * La maggior parte dei tool per ASP consente per comodità di usare variabili, ma le clausole devono poter essere trasformate in un numero finito di clausole ground
- * Esempio: in un problema di planning posso usare il termine `pickup(X)`, che verrà istanziato a `pickup(a)` e `pickup(b)`, se `a` e `b` sono gli unici blocchi esistenti

ASP vs Prolog

- * in ASP l'ordine dei letterali non ha alcuna importanza
- * Prolog è goal-directed, ASP no
- * La SLD-risoluzione del Prolog può portare a loop, mentre gli ASP solver non lo consentono
- * Prolog ha il CUT, ASP no

Negazione

- * Classica: attraversa :- -treno
 - * più forte: si attraversa solo se si può derivare che il treno non è in arrivo
- * Per fallimento: attraversa :- not treno
 - * si può attraversare in assenza di informazione esplicita sul treno in arrivo
- * In realtà, un letterale negato $\neg p$ non ha nessuna proprietà particolare
 - * Viene considerato come se fosse un nuovo atomo positivo, aggiungendo il vincolo :- p , $\neg p$.

Semantica

- * Answer set = modello minimale
- * Programma ASP privo di letterali not pi ha un unico modello minimale che è il suo answer set

```
p :- q.  
q.
```

Answer set= {p, q}

```
p :- q, r.  
q.
```

Answer set= {q}

Semantica

- * Ridotto P^S di un programma P rispetto ad un insieme di atomi S :
 - * rimuovi ogni regola il cui corpo contiene $\text{not } L$, per $L \in S$
 - * rimuovi tutti i $\text{not } L$ dai corpi delle restanti regole
- * P^S non contiene atomi con negazione per fallimento (per costruzione)
 - * ha un unico answer set
 - * se tale answer set coincide con S , allora S è un answer set per P

Semantica - Esempio

$p \text{ :- } a.$

$a \text{ :- } \text{not } b.$

$b \text{ :- } \text{not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Semantica - Esempio

$p \text{ :- } a.$

$a \text{ :- } \text{not } b.$

$b \text{ :- } \text{not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Insieme vuoto $\emptyset$

Semantica - Esempio

$p \text{ :- } a.$
 $a \text{ :- not } b.$
 $b \text{ :- not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Insieme vuoto $\emptyset$

Ridotto $P^{\emptyset}$

$p \text{ :- } a.$
 $a.$
 $b.$

Semantica - Esempio

$p \text{ :- } a.$
 $a \text{ :- not } b.$
 $b \text{ :- not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Insieme vuoto $\emptyset$

Ridotto $P^\emptyset$

$p \text{ :- } a.$
 $a.$
 $b.$

Answer set per $P^\emptyset = \{a, b, p\}$

Semantica - Esempio

$p \text{ :- } a.$
 $a \text{ :- not } b.$
 $b \text{ :- not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Insieme vuoto $\emptyset$

Ridotto $P^\emptyset$

$p \text{ :- } a.$
 $a.$
 $b.$

Answer set per $P^\emptyset = \{a, b, p\}$

Diverso da $\emptyset$, quindi non è un answer set per P

Semantica - Esempio

$p \text{ :- } a.$

$a \text{ :- } \text{not } b.$

$b \text{ :- } \text{not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Insieme $S=\{a\}$

Semantica - Esempio

$p \text{ :- } a.$
 $a \text{ :- not } b.$
 $b \text{ :- not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Insieme $S=\{a\}$

Ridotto P^S

$p \text{ :- } a.$
 $a.$
 $b \text{ :- not } a.$

Semantica - Esempio

$p \text{ :- } a.$
 $a \text{ :- not } b.$
 $b \text{ :- not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Insieme $S=\{a\}$

Ridotto P^S

$p \text{ :- } a.$
 $a.$
 $b \text{ :- not } a.$

Answer set per $P^S = \{a, p\}$

Semantica - Esempio

$p :- a.$
 $a :- \text{not } b.$
 $b :- \text{not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Insieme $S=\{a\}$

Ridotto P^S

$p :- a.$
 $a.$
 $b :- \text{not } a.$

Answer set per $P^S = \{a, p\}$

Diverso da $S=\{a\}$, quindi non è un answer set per P

Semantica - Esempio

```
p :- a.  
a :- not b.  
b :- not a.
```

Candidati: sottoinsiemi di $\{a, b, p\}$

Insieme $S=\{b\}$

Ridotto P^S

```
p :- a.  
a :- not b.  
b.
```

Answer set per $P^S = \{b\}$

Coincide con $S=\{b\}$, quindi è un answer set per P

Semantica - Esempio

$p \text{ :- } a.$

$a \text{ :- } \text{not } b.$

$b \text{ :- } \text{not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Answer sets: $\{b\}$

Semantica - Esempio

$p \text{ :- } a.$
 $a \text{ :- not } b.$
 $b \text{ :- not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Answer sets: $\{b\}$

Insieme $S=\{p\}$

Ridotto P^S

$p \text{ :- } a.$
 $a.$
 $b.$

Answer set per $P^S = \{a, b, p\}$

Diverso da $S=\{p\}$, quindi non è un answer set per P

Semantica - Esempio

$p \text{ :- } a.$
 $a \text{ :- not } b.$
 $b \text{ :- not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Answer sets: $\{b\}$

Insieme $S=\{a, b\}$

Ridotto P^S

$p \text{ :- } a.$
 $a \text{ :- not } b.$
 $b \text{ :- not } a.$

Answer set per $P^S = \emptyset$

Diverso da $S=\{a, b\}$, quindi non è un answer set per P

Semantica - Esempio

$p \text{ :- } a.$
 $a \text{ :- not } b.$
 $b \text{ :- not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Answer sets: $\{b\}$

Insieme $S = \{b, p\}$

Ridotto P^S

$p \text{ :- } a.$
 $a \text{ :- not } b.$
 $b.$

Answer set per $P^S = \{b\}$

Diverso da $S = \{b, p\}$, quindi non è un answer set per P

Semantica - Esempio

$p \text{ :- } a.$
 $a \text{ :- not } b.$
 $b \text{ :- not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Answer sets: $\{b\}$

Insieme $S=\{a, p\}$

Ridotto P^S

$p \text{ :- } a.$
 $a.$
 $b \text{ :- not } a.$

Answer set per $P^S = \{a, p\}$

Coincide con $S=\{a, p\}$, quindi è un answer set per P

Semantica - Esempio

$p \text{ :- } a.$

$a \text{ :- } \text{not } b.$

$b \text{ :- } \text{not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Answer sets: $\{b\}, \{a, p\}$

Semantica - Esempio

$p :- a.$
 $a :- \text{not } b.$
 $b :- \text{not } a.$

Candidati: sottoinsiemi di $\{a, b, p\}$

Answer sets: $\{b\}, \{a, p\}$

Insieme $S = \{a, b, p\}$

Ridotto P^S

$p :- a.$
 $a :- \text{not } b.$
 $b :- \text{not } a.$

Answer set per $P^S = \emptyset$

Diverso da $S = \{a, b, p\}$, quindi non è un answer set per P

Semantica

- * Nell'esempio ho due answer set distinti
- * I constraints servono per “filtrare” gli answer set
- * Esistono programmi che hanno un modello ma non hanno answer sets:

* Esempio:

```
p :- not p, d.  
d.
```

$\{p, d\}$ è un modello

Ogni answer set deve contenere d

Candidati: $\{d\}$ e $\{p, d\}$

Semantica

- * Nell'esempio ho due answer set distinti
- * I constraints servono per “filtrare” gli answer set
- * Esistono programmi che hanno un modello ma non hanno answer sets:

* Esempio:

```
p :- not p, d.  
d.
```

$\{p, d\}$ è un modello

Ogni answer set deve contenere d

Candidati: $\{d\}$ e $\{p, d\}$

$P^{\{d\}}$

```
p :- d.  
d.
```

Answer set = $\{d, p\} \neq \{d\}$

Semantica

- * Nell'esempio ho due answer set distinti
- * I constraints servono per “filtrare” gli answer set
- * Esistono programmi che hanno un modello ma non hanno answer sets:

* Esempio:

```
p :- not p, d.  
d.
```

$\{p, d\}$ è un modello

Ogni answer set deve contenere d

Candidati: $\{d\}$ e $\{p, d\}$

$P_{\{p, d\}}$

```
p :- not p, d.  
d.
```

Answer set = $\{d\} \neq \{p, d\}$

Soluzione di problemi

- * ASP: programmazione generate-and-test per risolvere un problema
- * Idea: descrivere un problema caratterizzandone le soluzioni
 - * ogni answer set del programma è una soluzione del problema
 - * l'ASP-solver trova una (o tutte) le soluzioni, nel momento stesso in cui calcola gli answer set del programma
- * Due fasi nella risoluzione di un problema:
 - * descrizione mediante regole di un insieme di soluzioni candidate (potenziali answer set).
 - * introduzione di vincoli (regole senza testa) che invalidano gli answer set che non sono realmente accettabili come soluzioni del problema

Aggregati

- * Selezionano answer set contenenti almeno/al massimo n/m elementi di un insieme
 - * answer set in cui ci sono da n a m atomi p
 - * i possibili valori vengono presi da q

$n \{ p(X) : q(X) \} m.$

- * Esempio: ogni variabile ha un solo tipo:

$1 \{ \text{ha_tipo}(V,T) : \text{tipo}(T) \} 1 \text{ :- variabile}(V).$

CLINGO

- * Gli esercizi ASP proposti nel corso possono essere svolti con Clingo
- * Free download al sito <https://potassco.org/clingo/>.
- * Le caratteristiche di Clingo sono descritte in dettaglio nella Guida Utente fornita sulla pagina moodle del corso